Python Programming Internship

Internship Completion Report

Muhammad Uzair

BS Electrical Engineering, PIEAS

Code Alpha • Python Programming Internship

Tasks Completed: 3

Date: July 5, 2026

0 Contents

0 Introduction

The following pages present the complete, well-documented Python code for all three projects completed during the Code Alpha internship. Each project is implemented as a standalone script and demonstrates core programming concepts such as conditionals, loops, file I/O, and user interaction. The code is presented in a clean, syntax-highlighted format for easy review.

1 Hangman Game

Overview

A classic word-guessing game. The player tries to identify a randomly selected word, letter by letter. After six incorrect guesses, the game ends and the hidden word is revealed.

Key Features

- Random word selection from a built-in list.
- Visual progress with underscores.
- Input validation (single alphabetic character, duplicate prevention).
- ASCII art displayed when the player loses.

Source Code

hangman.py

```python
import random      # Used to randomly select a word from the list

# List of possible secret words
words = ["apple", "football", "paris", "lion", "asad"]

# Randomly choose one word and convert it to lowercase
word = random.choice(words).lower()

# Create a list of underscores (_) equal to the length of the secret word
display = ["_"] * len(word)

# Total number of incorrect guesses allowed
attempts = 6

# Stores all letters already guessed by the player
guessed = []

# Welcome Screen
print("=" * 55)
print(" " * 14 + "WELCOME TO THE HANGMAN GAME")
print("=" * 55)
print("\nThe Goal: Save the Man by guessing the Secret Word.")
print("\nRules:")
print(">> Guess one letter at a time.")
print(">> You have 6 wrong attempts.")
print(">> Guess all the letters to win.")
print("-" * 55)

# Ask the user whether to start the game
start = input("Enter S to Start Game: ").upper()

# Start the game only if the user enters S
if start == "S":
    # Continue playing until attempts become zero
    while attempts > 0:
        # Display the current progress of the word
        print("\nWord:", " ".join(display))
        print("Attempts Left:", attempts)

        # Take one letter as input from the user
        guess = input("Guess a Letter: ").lower()

        # Validate that only one alphabet letter is entered
        if len(guess) != 1 or not guess.isalpha():
            print("Please enter only one letter.")
            continue

```

```
        # Check if the letter was already guessed
        if guess in guessed:
            print("You already guessed this letter.")
            continue

        # Store the guessed letter to prevent duplicate guesses
        guessed.append(guess)

        # Check whether the guessed letter exists in the secret word
        if guess in word:
            # Replace underscores with the correctly guessed letter
            for i in range(len(word)):
                if word[i] == guess:
                    display[i] = guess
            print("Correct Guess!")
        else:
            # Reduce attempts for an incorrect guess
            attempts -= 1
            print("Wrong Guess! Try Again.")

        # Check if the player has guessed the complete word
        if "_" not in display:
            print("\nCongratulations! You Won.")
            print("The Secret Word was:", word)
            break

        # Display Game Over message if no attempts are left
        if attempts == 0:
            print("\nGAME OVER! The Man is Hanged.")
            print("      _____")
            print("     |     |")
            print("     |     O")
            print("     |    /|\\")
            print("     |    / \\")
            print("  ___|___")
            print("\nThe Secret Word was:", word)
# If the user does not press S
else:
    print("Game Ended!")
```

2 Stock Portfolio Tracker

Overview

A simple portfolio tracker that asks the user for the stocks they own and calculates the total investment. The results are saved to a text file.

Key Features

- Pre-loaded dictionary of stock names and prices.
- User specifies the number of distinct stocks and their quantities.
- Investment per stock and total investment are displayed and written to `portfolio.txt`.
- Graceful handling of unsupported stock names.

Source Code

portfolio.py

```python
# Dictionary containing stock names and their prices
stock = {
    "Tesla": 160000,
    "NVIDIA": 760000,
    "Apple": 180000,
    "Google": 140000,
    "Microsoft": 230000,
    "Samsung": 260000
}

# Stores the total investment value
total = 0

# Create and open a file in write mode
file = open("portfolio.txt", "w")

# Write the heading in the file
file.write(f"{'=' * 10} STOCK PORTFOLIO TRACKER {'=' * 10}\n\n")
file.write("Stocks List:\n")

# Ask the user how many different stocks they own
print(">> How many different Stocks do you own?")
num = int(input(">> Enter amount of Stocks: "))

# Loop according to the number of stocks entered
for i in range(num):
    # Take stock name and quantity from the user
    name = input("\nEnter Stock Name: ")
    quantity = int(input("Enter Quantity: "))

    # Check whether the stock exists in the dictionary
    if name in stock:
        # Calculate investment for the current stock
        investment = stock[name] * quantity
        total += investment

        # Display stock details on the screen
        print(name, ":", quantity, "x", stock[name], "= $", investment)

        # Save stock details in the file
        file.write(name + " : " + str(quantity) + " x " + str(stock[name]) +
                   " = $" + str(investment) + "\n")
    else:
        # If stock is not available in the dictionary
        print("Stock not Found!")
        file.write(name + " : Stock not Found!\n")

```

```
# Display total investment
print("\nTotal Investment = $", total)

# Save total investment to the file
file.write("\nTotal Investment = $" + str(total) + "\n")
file.write("=" * 45)

# Close the file to save all changes
file.close()
print("\nPortfolio saved to portfolio.txt")
```

3 Flash Chatbot

Overview

A rule-based conversational agent named Flash. It responds to a set of predefined phrases and runs in an infinite loop until the user types “bye”.

Key Features

- Function `chat_bot()` processes user input and prints a matching response.
- Recognizes greetings, small talk, and personal questions.
- Default fallback message for unrecognised input.
- Clean welcome screen and exit condition.

Source Code

chatbot.py

```python
# Function to process the user’s input and generate a response
def chat_bot(chat):
    # Check the user’s message and respond accordingly
    if chat == "hello" or chat == "hi":
        print("Flash: Hello Uzair! Nice to see you.")
    elif chat == "how are you":
        print("Flash: I’m doing great! How about you, Uzair?")
    elif chat == "what is your name":
        print("Flash: My name is Flash.")
    elif chat == "tell me a quote":
        print("Flash: Have faith in yourself. Small steps every day lead to big achievements.")
    elif chat == "what is my name":
        print("Flash: Your name is Uzair.")
    elif chat == "who made you":
        print("Flash: I was made using Python by Uzair.")
    elif chat == "what can you do":
        print("Flash: I can answer simple questions and chat with you.")
    elif chat == "thank you" or chat == "thanks":
        print("Flash: You’re most welcome, Uzair!")
    elif chat == "good morning":
        print("Flash: Good morning, Uzair! Have a wonderful day.")
    elif chat == "good night":
        print("Flash: Good night, Uzair! Sweet dreams.")
    elif chat == "bye":
        print("Flash: Goodbye, Uzair! See you again.")
    # Default response for unrecognized input
    else:
        print("Flash: Sorry Uzair, I don’t understand that.")

# Display chatbot welcome screen
print("=" * 50)
print(" " * 12 + "Welcome to Flash ChatBot")
print("=" * 50)
print("\nFlash: Hello Uzair!")
print("Flash: Type ’bye’ whenever you want to leave.")

# Variable to store the user’s message
chat = ""

# Keep chatting until the user types "bye"
while chat != "bye":
    # Take input from the user and convert it to lowercase
    chat = input("\nUzair: ").lower()
    # Call the chatbot function to generate a response
    chat_bot(chat)
```

3 Skills Demonstrated

- **Core Python:** variables, conditionals (`if-elif-else`), loops (`while`, `for`), functions, and string manipulation.
- **Data structures:** lists, dictionaries, and their methods.
- **Input validation:** checking length, alphabetic characters, and membership.
- **File handling:** opening, writing, and closing text files for data persistence.
- **Randomisation:** using the `random` module.
- **User interaction:** clear console prompts and informative feedback.

3 Conclusion

All three projects were successfully implemented as per the internship requirements. Each script runs independently and demonstrates a solid understanding of fundamental programming concepts. The code is structured, readable, and ready to be extended with more advanced features such as persistent databases, graphical interfaces, or live data APIs.